

VERIQO -- RED FLAG RESPONSE REPORT

Assessment date: 28 August 2026

Baseline: VTECP-001 + CRE closure report -- commit 2ad92bc

Responds to: Red_Flag_yang_harus_dtuntaskan.docx -- an independent reviewer's response to the

prior closure report, naming three specific "red flags" and pushing back explicitly on the phrase "All 10 phases are complete" as an overstatement of platform readiness.

Assessment mode: local engineering qualification and reproducibility review. All code in this deliverable compiles, is tested, and was verified against the live repository.

EXECUTIVE VERDICT

The reviewer's three red flags are addressed one by one below. Two (LocalAnchorer, pkg/inference's scope) were already honestly documented in the prior report and required no code change -- the reviewer agreed with the architecture and asked only that the status stay visible in the gap register, which this report does explicitly. The third -- "Finding.ConfidenceBasis masih dapat dipalsukan" (still forgeable), named as the most serious of the three and a residual security/integrity gap rather than a footnote -- required a real code change and got one: a sealed AuthorizedFinding type that makes the reviewer's own requested pipeline shape a compile-time guarantee, not a documented convention.

The reviewer also asked for something the prior report did not provide: a genuine multi-stage integration test proving an inference actually passes through Model Registry -> Approval -> Policy -> Evidence Grounding -> Inference -> Output Validation -> Human Review -> Audit, "bukan hanya unit-test terhadap interface" (not just a unit test against the interface). That test now exists and passes, with an explicit, honest label on the one stage (Inference) that cannot be proven against a real external system in this environment.

Finally, the reviewer's own three-level framework -- Level 1 (Code Complete) / Level 2 (Engineering Verified) / Level 3 (Real-World Production Verified) -- is adopted here explicitly, replacing the prior report's less precise "All 10 phases are complete." Level 3 is not claimed. This report says so plainly, with the same gap-register table the reviewer's own document proposed.

RED FLAG 1: LOCALANCHORER IS A SIMULATOR

Reviewer's point: "Cryptographic anchoring architecture != real external anchoring." Saying "Evidence integrity is externally anchored" to an investor or customer is not yet a claimable, live production capability. Correct status: "Implemented -> internally verifiable, but: External anchoring -> NOT YET LIVE." Must stay in the final gap register.

Response: Agreed, and this was already the prior report's own position -- LocalAnchorer's package doc comment and every AnchorReceipt.AnchoredBy value it produces already say "LocalAnchorer(simulator, not a real external anchor)" verbatim, and the prior closure report's "Honest scope boundaries" section already stated this. No code change was needed; the reviewer's own two-line status framing is reproduced verbatim in this report's gap register below, so

it
stays visible rather than being buried in a code comment alone.

RED FLAG 2: PKG/INFERENCE IS NOT AN AI EXECUTION ENGINE

Reviewer's point: Correctly identified as a deliberate, correct architecture decision, not a weakness -- AI should propose/interpret/rank; VERIQO should verify/govern/trace/decide workflow

state. InferenceTrace is a governance layer, not a model engine. But one piece of follow-up work remained: "real model gateway integration test... untuk membuktikan bahwa model eksternal

benar-benar melewati Model Registry -> Approval -> Policy -> Evidence Grounding -> Inference -> Output
Validation -> Human Review -> Audit, bukan hanya unit-test terhadap interface."

Response: Built. TestModelGatewayFullPipeline
(test/integration/model_gateway_pipeline_test.go)
walks all eight named stages using the real package at each one:

Stage	Real package exercised	What the test actually checks
1. Model Registry	pkg/governance/lifecycle	Model registers in DRAFT
2. Approval	pkg/governance/lifecycle	VALIDATED->CALIBRATED->APPROVED->ACTIVE with a named approver, confirmed by re-reading the record
3. Policy	pkg/authz	Purpose-bound request allowed; the *same* request with no declared purpose is confirmed denied
4. Evidence Grounding	pkg/ontology + pkg/evidence/manifest	A real evidence object is walked DRAFT->FINALIZED, and the inference's InputHash is a hash *of* the finalized manifest's own hash -- grounding is checkable, not a label
5. Inference	pkg/inference	A trace is recorded -- explicitly labeled in the test's own doc comment as a real, governed, LOCAL call, not a real external LLM API, since none is configured in this environment
6. Output Validation	pkg/inference + pkg/insurance/finding	The trace's hash verifies, and the raw output alone is confirmed to decompose only as far as StatusCandidate -- never silently accepted as finished
7. Human Review	pkg/governance/hitl	Full Submit->Assign->Open->Act(APPROVE)->Execute workflow, producing a real GovernedOutcome naming the actual reviewer
8. Audit	pkg/platform/audit + hitl.Engine.VerifyChain	Both ledgers independently re-verified; a Merkle checkpoint is sealed and checked

What this does not and cannot honestly claim: network connectivity to a real external model provider. VERIQO has none configured in this sandboxed engineering environment, and claiming otherwise would violate this codebase's own anti-fabrication rule (VTECP-001 §58). What it does
prove, completely: the seven stages around inference are real, wired together in the reviewer's
own stated order, and enforced end to end -- the actual substance of "not just a unit test
against
the interface."

RED FLAG 3 (THE MOST SERIOUS): FINDING.CONFIDENCEBASIS WAS STILL FORGEABLE

Reviewer's point, verbatim in structure:

caller
v
manually creates Finding

v
injects ConfidenceBasis
v
Finding enters downstream system

VerifyFindingAgainstHypothesis existed, but the reviewer's question was sharper: "Apakah VERIQO memaksa semua Finding production melewati verification gate? Kalau jawabannya masih tidak, maka verification function ada, tetapi enforcement belum absolute." (Does VERIQO force every production Finding through the verification gate? If the answer is still no, the verification function exists but enforcement is not absolute.) The reviewer named the required shape explicitly:

Evidence -> Verification -> Hypothesis -> Finding Builder ->
Finding Verification Gate -> AUTHORIZED FINDING -> CRE / Dossier / Decision

with "Caller -> Finding" directly, skipping the gate, named as the one thing that must not be possible. The reviewer's own conclusion: "ini adalah satu gap yang menurut saya Claude harus tutup, bukan sekadar dokumentasikan" (this is a gap that must be closed, not just documented).

Response: closed, structurally, not by convention. pkg/insurance/cre gained a new file, authorized.go:

- AuthorizedFinding -- every field unexported. The only exported surface is read-only accessors (Finding(), HypothesisID(), AuthorizedAt(), AuthorizationHash(), IsZero()). There is no cre.AuthorizedFinding{finding: x} that compiles from outside this package -- unexported field names are not addressable across a package boundary in Go at all. The only value obtainable from outside pkg/insurance/cre is either the zero value (every accessor then returns empty/zero, which fails any check a careful consumer runs) or a value that genuinely came out of Authorize.
- Authorize is the only exported function that can populate one. It requires the Finding to already be at StatusFinding, then runs the full gate: VerifyFindingAgainstHypothesis against the *real* hypothesis, VerifyFindingProvenance against the *real* cited traces. Either failing returns the zero value and the error -- never a partially-authorized result.
- GenerateFindings -- the actual production entry point -- now returns []AuthorizedFinding, not []finding.Finding. Its signature changed: every call site in this repository (engine tests, both golden-case tests, the cross-system integration test, the adversarial test) was updated accordingly. This means "skip the gate" does not typecheck for any real caller of the engine going forward -- not merely "is discouraged in a doc comment."

The direct proof the reviewer asked for:
TestAuthorizeRefusesAHandForgedFindingEvenWhenInternallyConsistent constructs exactly the attack the reviewer's diagram describes -- a caller builds a finding.Finding by hand, injects a ConfidenceBasis the real hypothesis never earned, and recomputes the Finding's own hash so it stays internally self-consistent (exactly what a real forger capable of producing one would do). The forged Finding does structurally reach StatusFinding on its own terms --

proving that reaching StatusFinding alone was never sufficient proof, which is precisely why this gate exists as a separate step. Authorize still refuses it, because it re-derives the truth

from the real causation.HypothesisSet rather than trusting anything the Finding claims about itself. The same forged Finding is exercised again in the cross-file adversarial test (test/integration/cre_vtecp_adversarial_test.go), confirming the refusal holds from an external package's own call site too, not merely from inside pkg/insurance/cre's own test suite.

What is honestly still true after this fix: Authorize requires a caller to pass it a real *causation.HypothesisSet and the correct HypothesisID. If a caller constructs both a forged Finding *and* a forged HypothesisSet that happens to agree with it, Authorize would (correctly) accept it -- but at that point the caller has reconstructed a HypothesisSet whose SupportingEvidence/ContradictingEvidence genuinely say what the Finding claims, which is no longer forgery, it is the caller doing the real work BuildFinding would have done. The gate's actual guarantee is narrower and more honest than "no one can ever lie": it guarantees that an AuthorizedFinding's claims are *consistent with some HypothesisSet the caller can produce*, exactly as VerifyFindingAgainstHypothesis's own doc comment already stated before this round -- what changed is that a caller can no longer *skip* presenting one at all.

GAP REGISTER (REVIEWER'S OWN FRAMING, REPRODUCED AND UPDATED)

Area	Status	Note
---	---	---
Source code integrity	[GREEN]	unchanged
Unit testing	[GREEN]	unchanged
Integration testing	[GREEN]	strengthened this round (2 new integration tests)
Race safety	[GREEN]	unchanged, re-verified this round
Static analysis	[GREEN]	unchanged, re-verified this round
Structural guardrails	[GREEN]	unchanged, re-verified this round
Evidence manifest architecture	[GREEN]	unchanged
Ontology foundation	[GREEN]	unchanged
AI governance foundation	[GREEN]	unchanged
CRE integration	[GREEN]	unchanged
Finding verification	[GREEN]	foundation unchanged
Finding enforcement	[GREEN]	absolute, this round was [YELLOW] "perlu dipastikan" -- now closed via AuthorizedFinding / Authorize, see Red Flag 3 above
External anchoring	[YELLOW]	simulator unchanged, explicitly not claimed live -- see Red Flag 1
Real AI model execution	[YELLOW]	gateway proven, model call still local the 8-stage gateway is now proven end to end (Red Flag 2); the Inference stage itself remains a governed local call, honestly labeled, since no external model provider is configured in this environment
Real-world data validation	[RED]	not proven from this or any prior report in this engagement
Real-world insurance network	[RED]	not proven from this or any prior report in this engagement
Production customer environment	[RED]	not proven
End-to-end real case	[YELLOW]/[RED]	depends on evidence not yet available -- unchanged

Only one row moved this round: Finding enforcement, from [YELLOW] to [GREEN]. Every [RED] row remains [RED] -- this report does not claim otherwise, and no code change in this round could honestly move them,

since they require real external data, a real counterparty network, and a real production deployment, none of which exist in this engineering environment.

THE THREE-LEVEL FRAMEWORK (ADOPTED AS THE REVIEWER REQUESTED)

The reviewer's pushback on "All 10 phases are complete": that sentence should be read, and is now explicitly stated, as "all 10 engineering verification phases in the defined work package are complete" -- not as a claim of platform completion. Phase completion != platform completion. The reviewer's own three levels:

- Level 1 -- Code Complete. The targeted code has been written. [GREEN]
- Level 2 -- Engineering Verified. The code passes test, race, vet, integration, adversarial, and (across the wider repository) chaos and soak testing. [GREEN] -- and, per the reviewer's own assessment of the prior report, "sangat kuat" (very strong). This round adds one more genuine strengthening: the Finding authorization gate is now a structural guarantee, not a documented convention, and the model-gateway pipeline is proven end to end rather than asserted.
- Level 3 -- Real-World Production Verified. The code works against the real world -- real data, real actors, real workflows, real networks, real permissions, real external systems, and real operational conditions. [YELLOW], not final -- and this report does not claim otherwise. No claim in this report or any prior report in this engagement should be read as Level 3 evidence: every golden case, every "real" package this round exercises, and the model-gateway pipeline itself all run against synthetic, clearly-labeled test fixtures, not production data or a production deployment.

VERIFICATION PERFORMED

- gofmt -l across every file changed this round: clean.
- go build ./...: clean.
- go vet ./...: clean.
- go test -race on every changed/extended package (pkg/insurance/cre, pkg/insurance/finding, pkg/inference, pkg/authz, pkg/ontology, pkg/platform/audit, pkg/canonical/jcs, pkg/evidence/manifest, every pkg/governance/* subpackage, test/integration): clean.
- pkg/insurance/guardrails' seven repo-wide structural scans: clean.
- Full repository test suite (go test ./..., all ~150+ packages): run as this round's closing verification.
- 7 new tests this round (6 in pkg/insurance/cre/authorized_test.go, 1 in test/integration/model_gateway_pipeline_test.go), all passing on first run, plus every pre-existing test in every call site the GenerateFindings signature change touched, updated and re-verified passing.

FILES CHANGED THIS ROUND

3 commits, 2ad92bc..HEAD on claude/l99-gap-coverage-nv70zy:

pkg/insurance/cre/authorized.go	[new] AuthorizedFinding + Authorize
pkg/insurance/cre/authorized_test.go	[new] 6 tests
pkg/insurance/cre/engine.go	[changed] GenerateFindings now

authorizes
pkg/insurance/cre/engine_test.go [changed] updated call sites
test/integration/cre_golden_cases_test.go [changed] updated call sites
test/integration/cre_vtecp_adversarial_test.go [changed] updated call sites + stronger
assertion
test/integration/vtecp_cre_integration_test.go [changed] now demonstrates Authorize as
its final step
test/integration/model_gateway_pipeline_test.go [new] the 8-stage gateway proof
docs/VERIQO_RED_FLAG_RESPONSE_REPORT.{md, pdf} [new] this report

CONCLUSION

Of the three red flags, two were already honest and required no code change -- this report makes their status more visible, not different. The third and most serious -- Finding enforcement not being absolute -- is now closed structurally: a bare finding.Finding cannot reach any consumer that requires an AuthorizedFinding without passing through Authorize, and that gate re-derives the truth from real source data rather than trusting the Finding's own claims, proven against a hand-forged Finding specifically engineered to look self-consistent. The model-gateway pipeline the reviewer asked for now exists and passes, honestly labeled at its one unavoidable local-call boundary. Level 3 (Real-World Production Verified) is not claimed by this report, by the prior report, or by any test in this repository -- that distinction is now explicit rather than implied away by a phrase like "all phases complete."